

Multiplicação de matrizes e análise de desempenho

SSC0951 — Desenvolvimento de Código Otimizado

Atividade relativa à aula 3 • Coleta: 07/09/2026

Fernando Aleo Suaiden — Nº USP 12680836

Bruno Dalcantoni Cozac — Nº USP 13686323

1. O que foi feito

Foram comparadas quatro implementações seriais da multiplicação de matrizes, cada uma com alocação estática e dinâmica, totalizando os oito experimentos do enunciado [1]. O cálculo segue a aula 3 [2]: cada elemento da saída soma os produtos de uma linha de A por uma coluna de B.

Estática	Dinâmica	Técnica	Implementação
E1	E2	Simples	Laços $i \rightarrow j \rightarrow k$
E3	E4	Interchange	Troca para $i \rightarrow k \rightarrow j$
E5	E6	Unrolling	Duas operações por iteração de k
E7	E8	Tiling	Blocos de 32 em cada dimensão

Configuração. Matrizes 512×512 de float, entradas idênticas e compilação GCC 15.3.1 com -O0 em todas as versões. Unrolling e tiling são explícitos, sem OpenMP. As matrizes estáticas são contíguas; as dinâmicas usam um vetor de ponteiros e uma alocação por linha.

Máquina. AMD Ryzen 7 5700U (8 núcleos/16 CPUs lógicas), Fedora 43, kernel e perf 7.1.13. A L1 de dados tem 32 KiB por núcleo e a L2, 512 KiB. O programa usou uma thread fixada na CPU lógica 2. Frequência e boost permaneceram sob controle do sistema.

Coleta. Foram feitas dez rodadas com os oito experimentos em ordem sorteada: **80 medições**. O piloto foi separado. O tempo mede zeramento da saída e multiplicação. Os quatro contadores do perf foram ativados ao redor desse trecho, em modo usuário, excluindo alocação, geração das entradas e impressão, com pequeno custo de controle [3]. Todos tiveram cobertura de 100%, sem multiplexação. Nenhuma execução foi descartada.

Validação. As oito versões passaram em testes contra uma referência em double, com matrizes densas, identidade e zero, tamanhos ímpares, blocos incompletos e chamadas repetidas. As 80 medições tiveram o mesmo checksum.

Estatística. Para cada métrica e experimento, calculou-se média e intervalo de confiança de 95%: média $\pm 2,262157 \times s/\sqrt{10}$, com desvio padrão amostral e t de Student com nove graus de liberdade [4]. Nas tabelas, “ $\pm$ ” indica a margem desse intervalo. Os cálculos também foram conferidos em R.

2. Comparação dos tempos

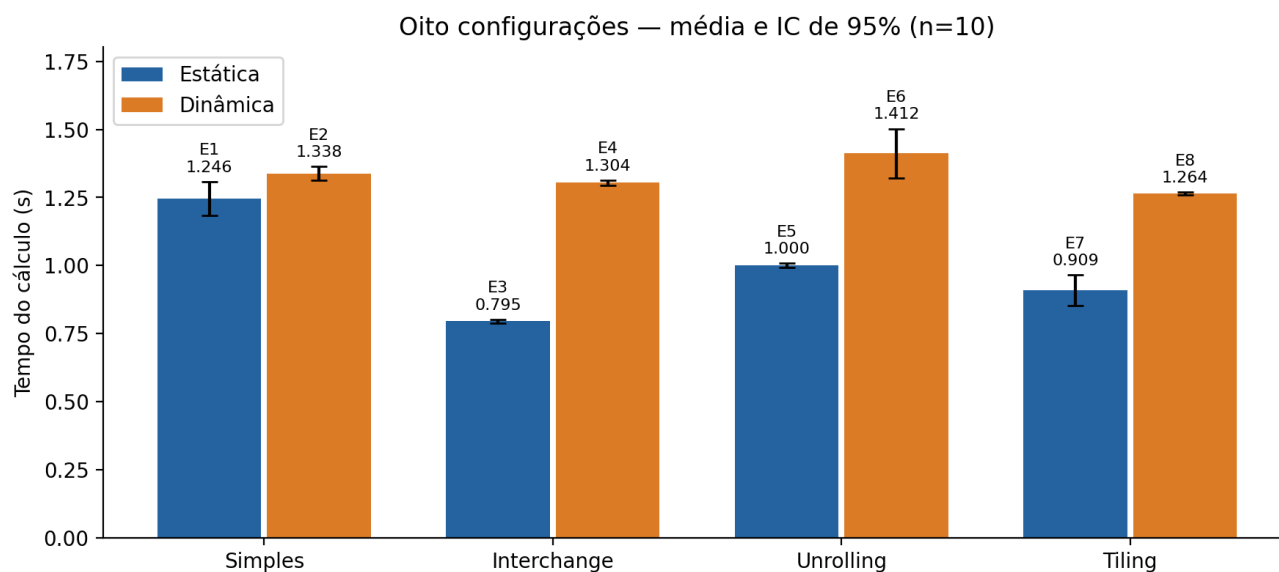


Figura 1 — Tempo médio do cálculo e intervalo de confiança de 95%; dez execuções por configuração.

Exp.	Técnica / alocação	Tempo (s): média ± IC95%	Razão simples / versão
E1	Simple / estática	1,246 ± 0,062	1,000×
E2	Simple / dinâmica	1,338 ± 0,025	1,000×
E3	Interchange / estática	0,795 ± 0,007	1,567×
E4	Interchange / dinâmica	1,304 ± 0,010	1,026×
E5	Unrolling / estática	1,000 ± 0,008	1,247×
E6	Unrolling / dinâmica	1,412 ± 0,091	0,948×
E7	Tiling / estática	0,909 ± 0,057	1,371×
E8	Tiling / dinâmica	1,264 ± 0,005	1,059×

Melhor resultado: interchange estático (E3), com $0,795 \pm 0,007$ s. O tempo médio foi 36,2% menor que o de E1, equivalente a uma razão de 1,57×. Na alocação estática, todas as técnicas tiveram média menor que a versão simples.

Na alocação dinâmica, tiling (E8) teve a menor média: redução de 5,6% em relação a E2. Unrolling dinâmico (E6) teve média 5,5% maior que E2. Os intervalos de E6 e E2 se sobrepõem; as razões apresentadas são comparações descritivas, não testes de significância.

A representação dinâmica adiciona acessos a ponteiros e muda a disposição das linhas. Em -O0, esses custos podem contribuir para a diferença de tempo. Os contadores medidos ajudam a interpretar o resultado, mas não isolam uma causa única.

3. Cache e desvios: resultados das oito versões

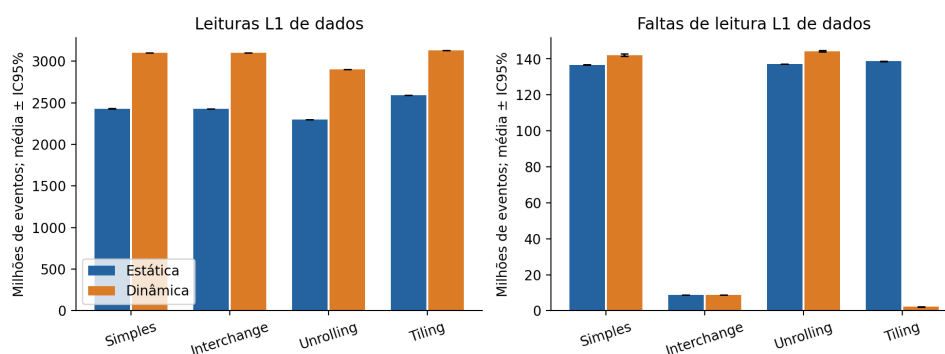


Figura 2 — Leituras e faltas de leitura na L1 de dados; médias e IC95%.

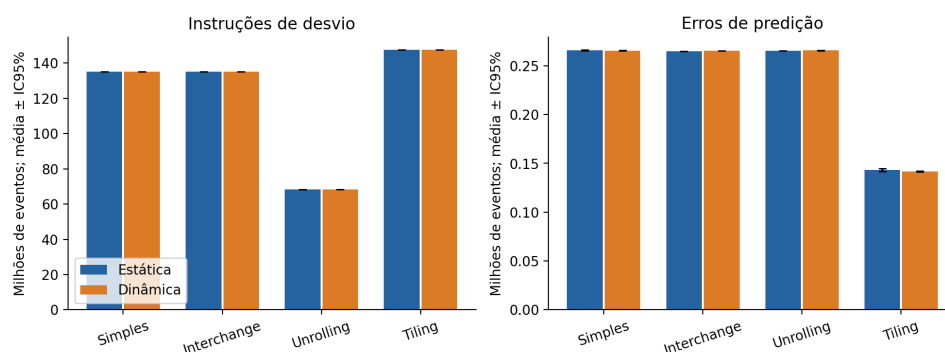


Figura 3 — Instruções de desvio e erros de predição; médias e IC95%.

Exp.	L1 loads (milhões)	L1 misses (milhões)	Branch instr. (milhões)	Branch misses (milhares)
E1	2.430,294 ± 1,395	136,723 ± 0,074	135,27209 ± 0,00007	265,920 ± 0,309
E2	3.101,304 ± 0,175	142,076 ± 0,625	135,27194 ± 0,00003	265,594 ± 0,227
E3	2.427,685 ± 0,126	8,700 ± 0,012	135,27163 ± 0,00004	264,771 ± 0,093
E4	3.098,745 ± 0,079	8,809 ± 0,019	135,27190 ± 0,00001	265,399 ± 0,115
E5	2.299,254 ± 1,613	137,130 ± 0,097	68,42512 ± 0,00001	265,319 ± 0,108
E6	2.902,800 ± 0,354	144,246 ± 0,400	68,42533 ± 0,00012	265,711 ± 0,325
E7	2.588,808 ± 0,225	138,738 ± 0,134	147,49065 ± 0,00038	143,274 ± 1,561
E8	3.128,553 ± 0,103	2,199 ± 0,092	147,49060 ± 0,00002	141,954 ± 0,464

Cache: interchange reduziu as faltas de L1 em cerca de 94% nas duas alocações, coerente com o acesso sequencial às linhas de B e da saída. Tiling teve comportamento diferente entre as alocações: E7 manteve muitas faltas e E8 reduziu-as fortemente. Conflitos de cache associados ao passo das linhas são uma hipótese, ainda não comprovada.

Desvios: unrolling reduziu as instruções de desvio em aproximadamente 49%, mas os erros de predição ficaram próximos de 265–266 mil. Portanto, menos desvios não significaram redução proporcional dos erros nem ganho garantido de tempo.

4. Influência da técnica, alocação e interação

Seguindo o material R [5], foram analisados dois planejamentos 2^2 : **E1-E4 para cache** (simples × interchange) e **E1,E2,E5,E6 para branch** (simples × unrolling). O segundo fator é estática × dinâmica. A interação mede se o efeito da técnica muda conforme a alocação.

O modelo é $y = q_0 + q_A \cdot A + q_B \cdot B + q_{AB} \cdot A \cdot B$, com níveis $-1/+1$. A influência do material é $100 \cdot q^2 / (q_A^2 + q_B^2 + q_{AB}^2)$. Ela reparte a variação entre as quatro médias; **não é o percentual de melhora de desempenho**. A adaptação em R usa `lm` e `anova` e reproduziu os resultados de Python.

Métrica	Técnica (%)	Alocação (%)	Interação (%)
L1 loads	0,001	99,999	0,000
L1 misses	99,916	0,044	0,040
Branch instr.	100,000	0,000	0,000
Branch misses	31,018	0,589	68,393

Tabela 3 — Influências calculadas como no script de apoio, considerando as quatro médias.

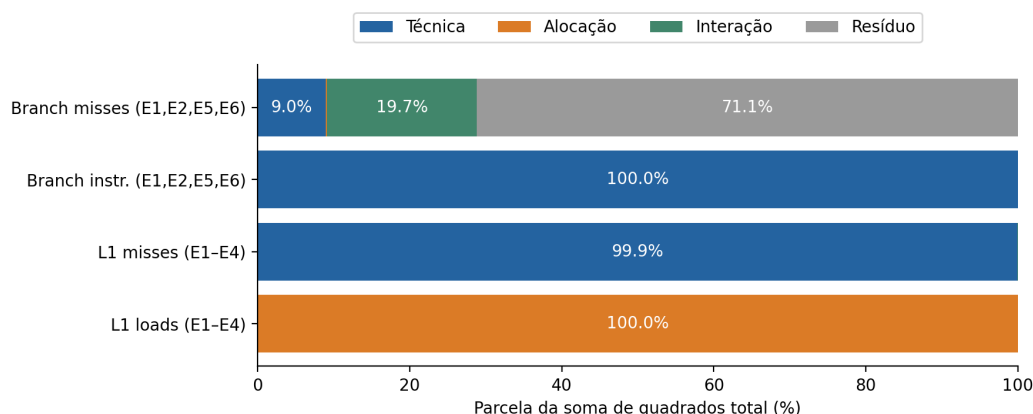


Figura 4 — Análise adicional das 40 observações de cada subconjunto, incluindo o erro entre repetições.

Métrica	Técnica (%)	Alocação (%)	Interação (%)	Resíduo (%)
L1 loads	0,001	99,998	0,000	0,001
L1 misses	99,912	0,044	0,040	0,004
Branch instr.	100,000	0,000	0,000	0,000
Branch misses	8,952	0,170	19,739	71,138

Tabela 4 — Com réplicas: $SQ_{fator} = 4 \cdot 10 \cdot q^2$; SQ_{erro} soma os desvios quadráticos dentro de cada combinação. Percentuais divididos pela soma de quadrados total. Valores muito pequenos podem arredondar para 0,000%.

Interpretação: a alocação domina a variação das leituras de L1; a técnica domina as faltas de L1 e o total de instruções de desvio. Em branch misses, 71,14% da variação total é resíduo entre execuções. Por isso, os 68,39% de interação da Tabela 3 não devem ser entendidos como grande ganho ou evidência forte de efeito.

5. Limitações e conclusão

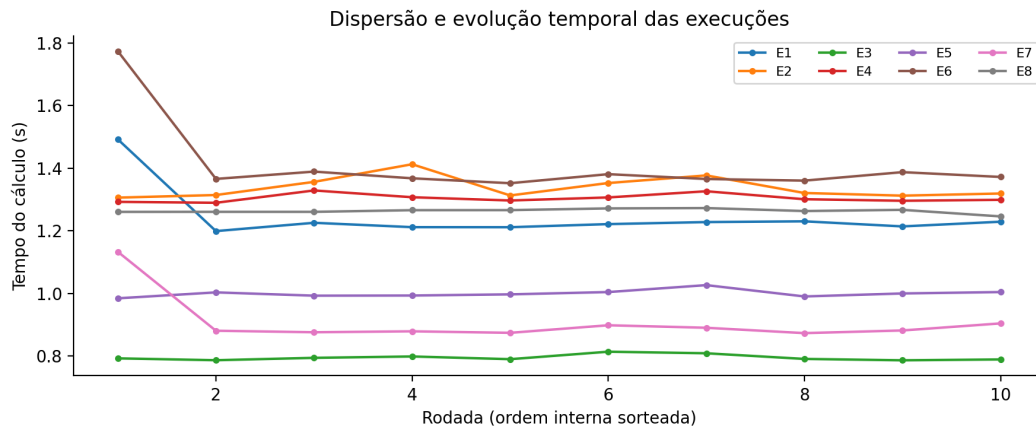


Figura 5 — Tempos por rodada. As primeiras observações mais lentas foram mantidas.

Limites da comparação. A máquina estava em uso geral, sem isolamento da CPU irmã e sem frequência fixa. Isso pode afetar tempos e contadores. Os intervalos t são individuais, sem ajuste para comparações múltiplas, e pressupõem independência e uma aproximação adequada para a distribuição da média. Dez observações não eliminam essas limitações.

O estudo usa um único tamanho de matriz e de bloco. A dimensão 512, potência de dois, pode acentuar conflitos de cache. Os resultados não devem ser generalizados para todas as dimensões, compiladores ou arquiteturas. A medição inclui zeramento e primeira escrita da saída, mas exclui `malloc/free`: avalia principalmente a disposição e o acesso às matrizes, não o custo de alocá-las.

Conclusão. Interchange estático apresentou o menor tempo médio e reduziu fortemente as faltas de L1. Unrolling quase dividiu por dois as instruções de desvio, mas não garantiu ganho na versão dinâmica. Tiling dependeu da disposição em memória. A análise das repetições mostrou que diferenças pequenas em branch misses exigem cautela.

Reprodução. O repositório preserva as 80 medições em `dados/coleta_2026-09-07/medicoes.csv`. As tabelas completas e os gráficos estão em `analise/`; `brutos.zip` contém as saídas originais e `fontes.zip`, o código usado. O piloto está separado em `historico/piloto.zip`. O comando **make relatorio**, na raiz, recalcula a análise e a conferência em R e gera este PDF e o `Anexo_tecnico.pdf`.

O anexo contém a metodologia completa e os gráficos de efeitos principais e interação. Os arquivos preservados têm verificação SHA-256; o relatório usa os dados medidos, sem descartar observações.

Referências

- [1] SSC0951. Atividade relativa à aula 3 (Profiling). Enunciado: `Atividade_1.pdf`.
- [2] BRUSCHI, Sarita Mazzini. 3ª aula — Profiling, páginas 14–17. `3aAula_Profiling.pdf`.
- [3] Linux perf. `perf-stat(1)`: controle dos eventos e coleta em modo usuário.
- [4] NIST/SEMATECH. [Confidence Limits for the Mean](#). IC bilateral t de Student.
- [5] SSC0951. Script R de apoio: `r.txt`, preservado em `Ex1/material_apoio/r_original.txt`. Adaptação: `Ex1/scripts/validar_analise.R`.